

AULA 08 • PROGRAMAÇÃO ORIENTADA A OBJETOS

Interfaces e classes abstratas

Definir **contratos** que as classes prometem cumprir — e
deixar o polimorfismo tratar muitos comportamentos por uma
única referência.

Curso de Java • P00 • Prof. Rob Silva

O caminho de hoje.

- 01 O problema: o que é uma abstração?
- 02 Classe abstrata e método abstrato
- 03 Interface — o contrato 100%
- 04 Quando usar cada uma
- 05 Aplicando na MegaStore (JAVA-104)
- 06 Tarefa da aula — ticket JAVA-108

Herdamos comportamento. E quando a base não faz sentido sozinha?

- Na `Aula 7`: `extends`, `@Override`, `super` e polimorfismo com `Animal`.
- Mas um `Animal` "genérico" não existe no mundo real — todo animal é de uma espécie.
- Queremos **obrigar** cada subclasse a definir o som, sem permitir criar um `Animal` solto.

Um molde que não vira objeto sozinho.

- Marcada com `abstract`: **não** pode ser instanciada diretamente.
- Pode ter atributos, métodos concretos e métodos abstratos (sem corpo).
- Cada subclasse **é obrigada** a implementar o que ficou abstrato.



A base define o contrato e o que já é comum.

```
public abstract class Animal {  
    protected String nome;  
  
    public Animal(String nome) {  
        this.nome = nome;  
    }  
  
    // método abstrato: sem corpo, obriga as subclasses  
    public abstract String emitirSom();  
  
    public String getNome() {  
        return nome;  
    }  
}
```

A palavra `abstract` aparece duas vezes: na **classe** (não instancia) e no **método** sem corpo (obriga quem herdar). `getNome()` já vem pronto.

As filhas cumprem a promessa com `@Override`.

```
public class Cachorro extends Animal {
    public Cachorro(String nome) {
        super(nome);
    }

    @Override
    public String emitirSom() {
        return "Au au!";
    }
}

public class Gato extends Animal {
    public Gato(String nome) {
        super(nome);
    }

    @Override
    public String emitirSom() {
        return "Miau!";
    }
}
```

Como `emitirSom()` era abstrato, implementá-lo é **obrigatório** — o compilador não deixa a subclasse concreta esquecer.

Para impedir o objeto "genérico" que não faz sentido.

- `new Animal("Bicho")`
agora **não compila** — e está correto.
- O tipo `Animal` continua servindo de referência comum para o polimorfismo.
- Reúso garantido (atributos, `getNome()`) e contrato garantido (`emitirSom()`).

Interface é contrato puro: só o "o quê", nunca o "como".

- Lista os métodos que uma classe **promete** ter — sem corpo, sem estado.
- A classe assina com `implements` e é obrigada a implementar tudo.
- Uma classe pode implementar **várias** interfaces ao mesmo tempo.

Pense em "capacidade": qualquer classe que saiba **calcular uma taxa** pode assinar o contrato — independente da sua hierarquia.

Duas linhas que definem um contrato inteiro.

```
public interface FormaPagamento {  
    double calcularTaxa(double valor);  
    String descricao();  
}
```

Nenhum corpo, nenhum atributo. Qualquer classe que `implements FormaPagamento` **tem** **que** oferecer `calcularTaxa(double)` e `descricao()`.

Classe abstrata × interface.

classe abstrata

Pode ter **estado** (atributos) e métodos concretos e abstratos.
Herança **única** (`extends`). Use no "é-um" parcial.

Regra de bolso: "é-um" parcial → **abstrata**; capacidade/contrato → **interface**. Na dúvida entre as duas, prefira a interface.

interface

Só **contrato** (Java básico): sem estado de instância.
Implementação **múltipla** (`implements A, B`). Use para "**capacidade**".

como decidir

É-um parcial, com código a compartilhar → **abstrata**.
Capacidade/contrato a garantir → **interface**.

POLIMORFISMO • EM LISTAS

Uma lista. Vários comportamentos.

Você guarda objetos diferentes numa `List<FormaPagamento>` e percorre com **um único** `for`. Cada um responde do seu jeito — sem `if` por tipo.

Registrar uma venda com forma de pagamento.

- O menu do `JAVA-104` (1 Produtos / 2 Categorias / 3 Relatórios / 0 Sair) ganha **Registrar venda**.
- Ao vender, o gerente escolhe a forma: **PIX**, **cartão** ou **boleto**.
- Cada forma calcula a **taxa** de um jeito — e o contrato `FormaPagamento` trata todas igual.

Cada forma assina o contrato do seu jeito.

```
public class Pix implements FormaPagamento {
    @Override
    public double calcularTaxa(double valor) {
        return 0.0;    // sem taxa
    }
    @Override
    public String descricao() {
        return "PIX (sem taxa)";
    }
}

public class CartaoCredito implements FormaPagamento {
    @Override
    public double calcularTaxa(double valor) {
        return valor * 0.04;    // 4%
    }
    @Override
    public String descricao() {
        return "Cartão de Crédito (4%)";
    }
}
```

Mesmos métodos, regras diferentes. A 3ª forma, `Boleto implements FormaPagamento`, seguiria a receita com taxa fixa (ex.: `return 2.50;`).

Uma List, um for, três comportamentos.

```
List<FormaPagamento> formas = new ArrayList<>();
formas.add(new Pix());
formas.add(new CartaoCredito());
formas.add(new Boleto());

double valor = 1000.0;
for (FormaPagamento fp : formas) {
    System.out.println(fp.descricao()
        + " -> taxa: R$ " + fp.calcularTaxa(valor));
}
```

A lista é do tipo `FormaPagamento` (a interface), mas dentro há `Pix`, `CartaoCredito` e `Boleto`. O laço é **um só** — cada objeto resolve a própria taxa.

Três ideias para levar para casa.

classe abstrata

Molde incompleto que não instancia. Traz estado e código comum, mas obriga as filhas a implementar o resto.

interface

Contrato puro com `implements`. Sem estado, múltipla. Garante capacidades sem amarrar a hierarquia.

polimorfismo em listas

Uma `List` do tipo do contrato, um `for` só. Cada objeto executa a sua versão — sem `if` por tipo.

JAVA-108

MÉDIO

TO DO

BACKEND

EVOLUI: JAVA-104

Registrar venda com formas de pagamento

HISTÓRIA DO USUÁRIO

Como gerente da MegaStore, quero registrar a venda de um produto escolhendo a forma de pagamento (**PIX**, **cartão** ou **boleto**), para que o sistema calcule a taxa cobrada em cada caso.

CONTEXTO

O menu do estoque ganhará uma opção de "**Registrar venda**". Cada forma de pagamento calcula a taxa de um jeito. Use um **contrato comum** para tratar todas igual — evoluindo o sistema do JAVA-104 .

JAVA-108

MÉDIO

PRONTO QUANDO TODOS OS ITENS PASSAREM

CRITÉRIOS DE ACEITE

- ☒ Criar a interface `FormaPagamento` com `calcularTaxa(double)` e `descricao()`.
- ☒ Implementar pelo menos `Pix`, `CartaoCredito` e `Boleto` com `@Override` nos dois métodos.
- ☒ Guardar as formas numa `List<FormaPagamento>` e percorrer com `for-each` chamando os métodos polimorficamente.
- ☒ Nenhuma classe pode ser instanciada a partir de tipo abstrato/interface diretamente — só das classes concretas.

DESAFIO EXTRA

Adicione uma nova forma (ex.: `ValeAlimentacao` ou um PIX com desconto) **sem alterar o laço** que percorre a lista.

#megastore

#java-104

#interfaces

#entrega: próxima aula

Resolva usando só interfaces, classes abstratas e polimorfismo em listas — nada além da Aula 8.

